



Технології графічного процесінгу

Керування пам'яттю в CUDA

Практична робота #2

Виконав(ла):
Олександр Ковальов
Група:
ІМ-51мн
Курс:
1

28 березня 2026 р.

1 Порівняльний аналіз підходів до управління пам'яттю в CUDA

1.1 Результати експериментів

Експерименти проводилися на апаратному забезпеченні, що включає процесор Intel Core i5-1235U та графічний прискорювач NVIDIA GeForce RTX 2050 з 4 ГБ відеопам'яті.

```
xairaven@laptop ~
$ fastfetch
      ,,:,,,:.
      ,,:ccccccccc;.,,
      ,:cccccccccccccccccccc;
      ,:cccccccccccccccccccc;.
      ,:cccccccccccc; .dddl; :cccccc;.
      ,:cccccccccccccc;0MMK00XMMw;cccccc;.
      ,:cccccccccccccc;KMMc;cc;XMMc;cccccc;.
      ,:cccccccccccccc;MMM;cc;WW;:cccccccc;
      ,:cccccccccccccc;MMM;:cccccccccccccccc;
      ,:cccccccc;ox000o;MMM000k;:cccccccccccc;
      ,:cccccc;0MMKxdd;:MMMkddc;:cccccccccccc;
      ,:cccc;XMO';:ccc;MMM;:cccccccccccccccc;
      ,:cccc;MMo;cccc;:MMW;:cccccccccccccccc;
      ,:cccc;0MNC.ccc.XMMd;cccccccccccccccc;
      ,:cccc;dnMWXXXWMO;:cccccccccccccccc;
      ,:cccccccc;.odl;:cccccccccccccc;.,,
      ,:cccccccccccccccccccccccccccc;'.
      ,:cccccccccccccccccccccccccc;.,,
      ,:cccccccccccccccccccc;.,,.,,

xairaven@laptop
-----
OS: Fedora Linux 43 (KDE Plasma Desktop Edition) x86_64
Host: Aspire A515-57G (V1.28)
Kernel: Linux 6.19.9-200.fc43.x86_64
Uptime: 3 hours, 15 mins
Packages: 8046 (rpm), 19 (flatpak-system), 29 (flatpak-user)
Shell: zsh 5.9
Display (BOE0A56): 1920x1080 @ 1.25x in 16", 60 Hz [Built-in]
DE: KDE Plasma 6.6.3
WM: KWin (Wayland)
WM Theme: Breeze
Theme: Breeze (Dark) [Qt], Breeze [GTK3]
Icons: breeze-dark [Qt], breeze-dark [GTK3/4]
Font: Segoe UI [10pt] [Qt], Segoe UI [10pt] [GTK3/4]
Cursor: breeze (24px)
Terminal: konsole 25.12.3
CPU: 12th Gen Intel(R) Core(TM) i5-1235U (12) @ 4.40 GHz
GPU 1: NVIDIA GeForce RTX 2050 [Discrete]
GPU 2: Intel Iris Xe Graphics @ 1.20 GHz [Integrated]
Memory: 6.54 GiB / 15.31 GiB (43%)
Swap: 0 B / 24.00 GiB (0%)
Disk (/): 51.27 GiB / 171.20 GiB (30%) - ext4
Disk (/home): 186.96 GiB / 278.82 GiB (67%) - ext4
Local IP (wlp0s20f3): 192.168.0.106/24
Battery (AP19B5L): 100% [AC Connected]
Locale: en_US.UTF-8
```

Рис. 1: Інформація про апаратне забезпечення.

Відеокарта побудована на архітектурі Ampere та має обчислювальну здатність 8.6. Для збору метрик продуктивності застосовувався профілювальник NVIDIA Nsight Systems у середовищі CUDA 13.0.

```
xairaven@laptop ~
$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2026 NVIDIA Corporation
Built on Mon_Mar_02_09:52:23_PM_PST_2026
Cuda compilation tools, release 13.2, V13.2.51
Build cuda_13.2.r13.2/compiler.37434383_0

xairaven@laptop ~
$ nvidia-smi
Fri Mar 27 23:16:44 2026

+-----+
| NVIDIA-SMI 580.126.18                Driver Version: 580.126.18    CUDA Version: 13.0     |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap       |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0    NVIDIA GeForce RTX 2050           Off          | 00000000:01:00:0 Off |                  N/A |
| N/A   43C    P8              2W / 30W   | 0MiB / 4096MiB |      0%    Default  |
|=====+=====+
|                  N/A                  |                  N/A |                  N/A |
+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                  | GPU Memory |
| ID     ID                               |            | Usage      |
|=====+=====+
| No running processes found |
+-----+
```

Рис. 2: Версія компілятора, CUDA.

Так як остання версія компілятора з репозиторію Fedora розрахована під CUDA 13.2, а поточний драйвер підтримує лише CUDA 13.0, при компіляції використовувався параметр `-arch=sm_86`, який точно встановлює архітектуру для компіляції.

```
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ nvcc ./01-vector-add-explicit-memory.cu -o explicit -arch=sm_86
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ nvcc ./02-vector-add-unified-memory.cu -o unified -arch=sm_86
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ nvcc ./03-vector-add-shared-memory.cu -o shared -arch=sm_86
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ ./explicit
Success! All values calculated correctly.
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ ./unified
Success! All values calculated correctly.
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$ ./shared
Success! All values calculated correctly.
xairaven@laptop ~/Files/University/10/GPT/Lab2/code
$
```

Рис. 3: Компіляція коду та запуск відповідних бінарних файлів.

Додавалися два вектори розміром понад 33 мільйони елементів, кожен з яких займав близько 134 МБ пам'яті.

У таблиці 1 наведено порівняння часу виконання обчислювального ядра та часу передачі результатуючих даних з відеокарти на центральний процесор для трьох досліджуваних підходів. Результати були отримані за допомогою команди `nsys profile -stats=true <binary-file>`

Табл. 1: Порівняння продуктивності різних підходів до управління пам'яттю.

Підхід	Час виконання ядра (мс)	Час копіювання D2H (мс)	Характер копіювання
Явне виділення пам'яті	4.47	47.0	Один монолітний блок
Уніфікована пам'ять	4.45	20.5	64 блоки по 2 МБ
Спільна пам'ять	4.43	46.1	Один монолітний блок

Час виконання самого обчислювального ядра залишається практично незмінним (в межах 4.43 - 4.47 мс) незалежно від обраного підходу до управління пам'яттю. Цей результат є цілком очікуваним, оскільки алгоритм додавання векторів відноситься до класу задач, що обмежуються пропускну здатністю пам'яті. Арифметична інтенсивність такої операції є вкрай низькою: на одну математичну операцію додавання припадає аж три звернення до глобальної пам'яті (зчитування елементів з масивів А та В, а також запис результату в масив С). За таких умов обчислювальні блоки мультипроцесорів більшу частину часу простоюють в очікуванні даних, тому зміна підходу до виділення пам'яті не може прискорити самі обчислення. Використання спільної пам'яті як проміжного кешу не дало жодного приросту продуктивності. Це пояснюється тим, що кожен потік звертається до свого елемента даних лише один раз. Відсутність повторного використання даних повністю нівелює головну перевагу спільної пам'яті. Більше того, завантаження даних у спільну пам'ять вимагає додаткових машинних інструкцій та обов'язкової синхронізації потоків всередині блоку за допомогою бар'єру, що додає накладних витрат.

Найбільш цікавим результатом експерименту є суттєва різниця у часі копіювання масиву результатів з відеопам'яті на центральний процесор. Традиційне явне копіювання та підхід зі спільною пам'яттю продемонстрували час близько 46-47 мілісекунд, передаючи весь масив обсягом 134.2 МБ одним монолітним блоком. Натомість використання уніфікованої пам'яті з механізмом асинхронної попередньої вибірки дозволило скоротити цей час більш ніж удвічі — до 20.5 мілісекунд. Драйвер NVIDIA автоматично розбив загальний обсяг даних на 64 окремі сторінки по 2 мегабайти. Таке фрагментоване пересилання дозволяє значно ефективніше утилізувати шину PCI Express, забезпечуючи краще конвеєризування операцій введення-виведення та уникаючи тривалих блокувань шини великими порціями даних.

1.2 Допомога

За основу для виконання роботи було взято базовий код з відкритого репозиторію <https://github.com/lintenn/cudaAddVectors-explicit-vs-unified-memory>. Також у групі з одногрупниками обговорювалася ідея використання Distributed Shared Memory для оптимізації. Ця концепція виявилася незастосовною для поточного обладнання, оскільки технологія Thread Block Clusters підтримується лише в архітектурі Норрег, тоді як наявна відеокарта RTX 2050 побудована на архітектурі Ampere. Це зумовило необхідність використання класичної спільної пам'яті на рівні окремого блоку.

1.3 Висновки

У ході виконання лабораторної роботи було проведено детальне порівняння трьох методів управління пам'яттю в технології CUDA на прикладі класичної задачі додавання векторів великої розмірності. Результати дослідження підтверджують, що швидкість виконання такого обчислювального ядра визначається виключно фізичною межею пропускної здатності пам'яті відеокарти, а не обраним програмним інтерфейсом виділення ресурсів. Обчислювальні потоки лінійно опрацьовують масиви, що не залишає простору для оптимізації самої арифметики.

Експеримент зі спільною пам'яттю наочно продемонстрував важливе архітектурне правило роботи з графічними прискорювачами. Застосування програмно керованого кешу першого рівня виправдане лише для тих алгоритмів, де наявне багаторазове використання одних і тих самих даних різними потоками в межах одного блоку (наприклад, при множенні матриць або згортці зображень). У потокових алгоритмах, де кожен елемент зчитується лише раз, перенесення даних у спільну пам'ять не зменшує навантаження на глобальну шину пам'яті, а лише додає накладні витрати на синхронізацію потоків, не приносячи жодного корисного ефекту.

Найвагомішим спостереженням став аналіз операцій передачі даних між пристроями. Порівняння часу копіювання продемонструвало беззаперечну перевагу уніфікованої пам'яті. Завдяки механізмам розбиття на сторінки за запитом та асинхронної вибірки, драйвер оптимізує транзакції через шину PCIe, розбиваючи монолітний запит на серію менших пересилань по 2 МБ. Це забезпечує понад двократне зменшення часу копіювання результатів порівняно з явним синхронним пересиланням функцією копіювання. Отримані результати дозволяють зробити висновок, що сучасна підсистема уніфікованої пам'яті NVIDIA не лише спрощує написання коду за рахунок єдиного адресного простору, але й здатна забезпечувати значно вищу продуктивність пересилання даних завдяки прихованим внутрішнім оптимізаціям драйвера.

2 Лістинг.

Лістинг 1: Додавання векторів з явним управлінням пам'яттю.

```
#include <stdio.h>

__global__
void initWith(float num, float *a, int N)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for(int i = index; i < N; i += stride)
    {
        a[i] = num;
    }
}

__global__
void addVectorsInto(float *result, float *a, float *b, int N)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for(int i = index; i < N; i += stride)
    {
        result[i] = a[i] + b[i];
    }
}

void checkElementsAre(float target, float *vector, int N)
{
    for(int i = 0; i < N; i++)
    {
        if(vector[i] != target)
        {
            printf("FAIL: vector[%d] - %0.0f does not equal %0.0f\n", i, vector[i], target);
            exit(1);
        }
    }
    printf("Success! All values calculated correctly.\n");
}

int main()
{
    int deviceId;
    int numberOfSMs;

    cudaGetDevice(&deviceId);
    cudaDeviceGetAttribute(&numberOfSMs, cudaDevAttrMultiProcessorCount, deviceId);

    const int N = 2<<24;
    size_t size = N * sizeof(float);
```

```
//float *a;
//float *b;
float *c;

//a = (float *) malloc(size);
//b = (float *) malloc(size);
c = (float *) malloc(size);

float *da;
float *db;
float *dc;

cudaMalloc(&da, size);
cudaMalloc(&db, size);
cudaMalloc(&dc, size);

size_t threadsPerBlock;
size_t numberOfBlocks;

threadsPerBlock = 256;
numberOfBlocks = 32 * numberOfSMs;

cudaError_t addVectorsErr;
cudaError_t asyncErr;

initWith<<<<numberOfBlocks, threadsPerBlock>>>>(3, da, N);
initWith<<<<numberOfBlocks, threadsPerBlock>>>>(4, db, N);
initWith<<<<numberOfBlocks, threadsPerBlock>>>>(0, dc, N);

addVectorsInto<<<<numberOfBlocks, threadsPerBlock>>>>(dc, da, db, N);
// Les pasamos los punteros de device (da,db,dc) y no de host (a,b,c)

addVectorsErr = cudaGetLastError();
if(addVectorsErr != cudaSuccess) printf("Error:_%s\n", cudaGetErrorString(a));

asyncErr = cudaDeviceSynchronize();
if(asyncErr != cudaSuccess) printf("Error:_%s\n", cudaGetErrorString(asyncErr));

//cudaMemcpy(a, da, size, cudaMemcpyDeviceToHost);
//cudaMemcpy(b, db, size, cudaMemcpyDeviceToHost);
cudaMemcpy(c, dc, size, cudaMemcpyDeviceToHost);

checkElementsAre(7, c, N);

cudaFree(da);
cudaFree(db);
cudaFree(dc);

//free(a);
//free(b);
free(c);
}
```

Лістинг 2: Додавання векторів з використанням уніфікованої пам'яті.

```
#include <stdio.h>

__global__
void initWith(float num, float *a, int N)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for(int i = index; i < N; i += stride)
    {
        a[i] = num;
    }
}

__global__
void addVectorsInto(float *result, float *a, float *b, int N)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for(int i = index; i < N; i += stride)
    {
        result[i] = a[i] + b[i];
    }
}

void checkElementsAre(float target, float *vector, int N)
{
    for(int i = 0; i < N; i++)
    {
        if(vector[i] != target)
        {
            printf("FAIL: _vector[%d] _ _%0.0f_does_not_equal_%0.0f\n", i, target);
            exit(1);
        }
    }
    printf("Success! _All_values_calculated_correctly.\n");
}

int main()
{
    int deviceId;
    int numberOfSMs;

    cudaGetDevice(&deviceId);
    cudaDeviceGetAttribute(&numberOfSMs, cudaDevAttrMultiProcessorCount, deviceId);

    const int N = 2<<24;
    size_t size = N * sizeof(float);

    float *a;
    float *b;
```

```

    float *c;

    cudaMallocManaged(&a, size);
    cudaMallocManaged(&b, size);
    cudaMallocManaged(&c, size);

    cudaMemLocation devLoc;
    devLoc.type = cudaMemLocationTypeDevice;
    devLoc.id = deviceId;

    cudaMemPrefetchAsync(a, size, devLoc, 0);
    cudaMemPrefetchAsync(b, size, devLoc, 0);
    cudaMemPrefetchAsync(c, size, devLoc, 0);

    size_t threadsPerBlock;
    size_t numberOfBlocks;

    threadsPerBlock = 256;
    numberOfBlocks = 32 * numberOfSMs;

    cudaError_t addVectorsErr;
    cudaError_t asyncErr;

    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(3, a, N);
    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(4, b, N);
    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(0, c, N);

    addVectorsInto<<<<numberOfBlocks, threadsPerBlock>>>>(c, a, b, N);

    addVectorsErr = cudaGetLastError();
    if(addVectorsErr != cudaSuccess) printf("Error: %s\n", cudaGetErrorString(addVectorsErr));

    asyncErr = cudaDeviceSynchronize();
    if(asyncErr != cudaSuccess) printf("Error: %s\n", cudaGetErrorString(asyncErr));

    // Define the memory location for the CPU
    cudaMemLocation hostLoc;
    hostLoc.type = cudaMemLocationTypeHost;
    hostLoc.id = 0;

    // Prefetch result to CPU on the default stream (0)
    cudaMemPrefetchAsync(c, size, hostLoc, 0);

    checkElementsAre(7, c, N);

    cudaFree(a);
    cudaFree(b);
    cudaFree(c);
}

```

Лістинг 3: Додавання векторів з використанням спільної пам'яті.

```
#include <stdio.h>
```

```
// Kernel to initialize array elements
```



```
__global__
void initWith(float num, float *a, int N)
{
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for(int i = index; i < N; i += stride)
    {
        a[i] = num;
    }
}

// Kernel that uses shared memory as a tile cache
__global__
void addVectorsIntoShared(float *result, float *a, float *b, int N)
{
    // Allocate dynamic shared memory for the current block
    extern __shared__ float shared_mem[];

    // Split shared memory into two logical arrays for inputs
    float* s_a = shared_mem;
    float* s_b = &shared_mem[blockDim.x];

    int index = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    int tid = threadIdx.x;

    for(int i = index; i < N; i += stride)
    {
        // Load elements from global memory to shared memory cache
        s_a[tid] = a[i];
        s_b[tid] = b[i];

        // Synchronize to ensure all elements are loaded
        __syncthreads();

        // Compute the sum using cached data
        float temp_sum = s_a[tid] + s_b[tid];

        // Synchronize before writing and moving to the next iteration
        __syncthreads();

        // Write the computed sum back to global memory
        result[i] = temp_sum;
    }
}

// Function to verify the results
void checkElementsAre(float target, float *vector, int N)
{
    for(int i = 0; i < N; i++)
    {
        if(vector[i] != target)
        {

```

```

        printf("FAIL: _vector[%d]_-%0.0f_does_not_equal_%0.0f\n", i, i, i);
        exit(1);
    }
}
printf("Success!_All_values_calculated_correctly.\n");
}

int main()
{
    int deviceId;
    int numberOfSMs;

    cudaGetDevice(&deviceId);
    cudaDeviceGetAttribute(&numberOfSMs, cudaDevAttrMultiProcessorCount, deviceId);

    const int N = 2<<24;
    size_t size = N * sizeof(float);

    float *c = (float *) malloc(size);

    float *da;
    float *db;
    float *dc;

    cudaMalloc(&da, size);
    cudaMalloc(&db, size);
    cudaMalloc(&dc, size);

    size_t threadsPerBlock = 256;
    size_t numberOfBlocks = 32 * numberOfSMs;

    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(3, da, N);
    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(4, db, N);
    initWith<<<<numberOfBlocks, threadsPerBlock>>>>(0, dc, N);

    // Calculate required shared memory size for two arrays per block
    size_t sharedMemSize = 2 * threadsPerBlock * sizeof(float);

    addVectorsIntoShared<<<<numberOfBlocks, threadsPerBlock, sharedMemSize>>>>(da, db, dc);

    cudaError_t addVectorsErr = cudaGetLastError();
    if(addVectorsErr != cudaSuccess) printf("Error:_%s\n", cudaGetErrorString(addVectorsErr));

    cudaError_t asyncErr = cudaDeviceSynchronize();
    if(asyncErr != cudaSuccess) printf("Error:_%s\n", cudaGetErrorString(asyncErr));

    cudaMemcpy(c, dc, size, cudaMemcpyDeviceToHost);

    checkElementsAre(7, c, N);

    cudaFree(da);
    cudaFree(db);
    cudaFree(dc);
}

```

```
        free(c);  
        return 0;  
    }
```